Programming With C

Language ?

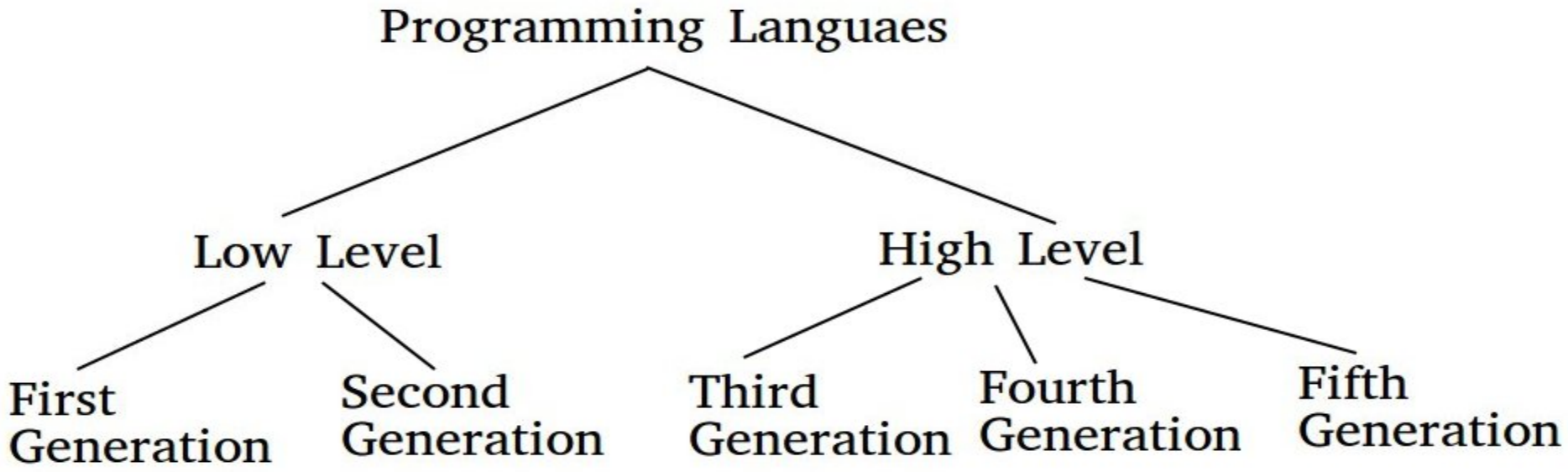
- Natural
- Formal
- Context Free
- Programming Language

Levels Of Programming Languages

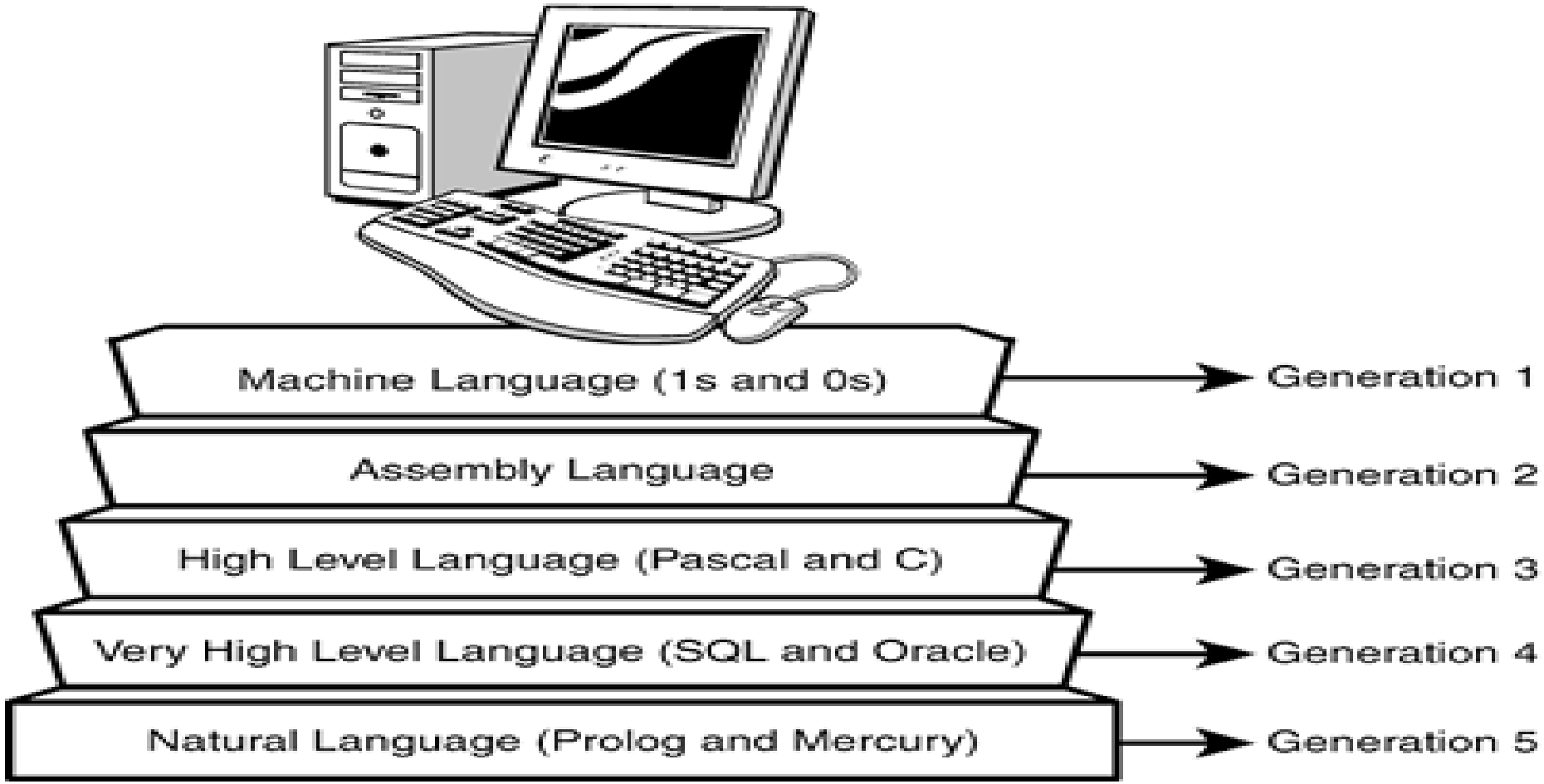
1 low level : machine language , assembly language

2 Middle level languages : Fortran , Cobol , pascal , C

3 High level languages : SQL, Prolog , Lisp ,



Compilers Vs Interpreters



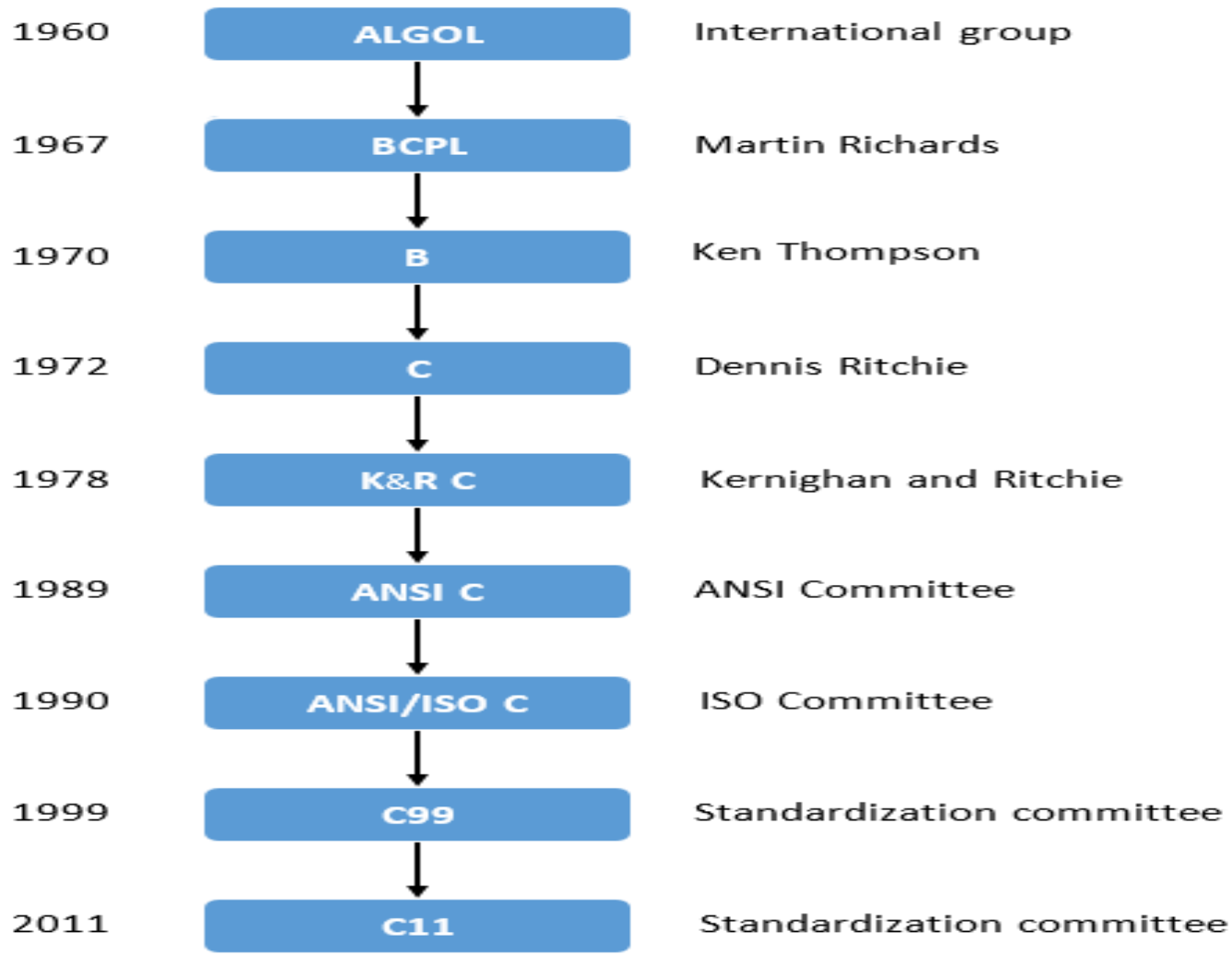
Generation Of Programming Languages

History of Programming Languages

- *1940s: Von Neumann*
- *1950s: First Programming Language*
- *1960s: Basic Programming (B Language)*
- *1970s: C Language(Simplicity, Abstraction,Study)*
  by Dennis Ritchie
- *1975s: C++ Language(object Oriented, Logic Programming)*
  by Benjamin Strawstroup
- *1980s:GUI Programming*
- *1990s: Visual Basic by Microsoft and Java by James Gosling*
  at Sun MicroSystems
- *1992: Platform Indepedent Language(Java)*
- *1999: Dot Net Platform By Microsoft*
- *2000s: Scripting, Web Programming Like PHP,Asp.net,Javascript,VBScript*
- *2010s: Parallel Computing, Concurrency*

By: Naveen Mishra
at www.webncc.in

History Of C


1960
ALGOL
International group
1967
BCPL
Martin Richards
1970
B
Ken Thompson
1972
C
Dennis Ritchie
1978
K&R C
Kernighan and Ritchie
1989
ANSI C
ANSI Committee
1990
ANSI/ISO C
ISO Committee
1999
C99
Standardization committee
2011
C11
Standardization committee

Developed in 1972 at AT&T bell labs

- GPPL
- Data Types
- UNIX
- ANSI American national standard Institute
- ASCII

Advantages of C

- Why C
- System programing
- Networking
- Virus and Anti virus
- Fire walls
- Compiler Designing
- Middle level Programming (benefits of both)
- No competitor at that time

- Data Types
- Data structures
- Functions, Structures and unions
- For Loop
- Do While Loop
- Switch Case
- Pointers
- File handling
- Addressing
- OS designing Algorithms
- Loaders and Linkers
- ETC

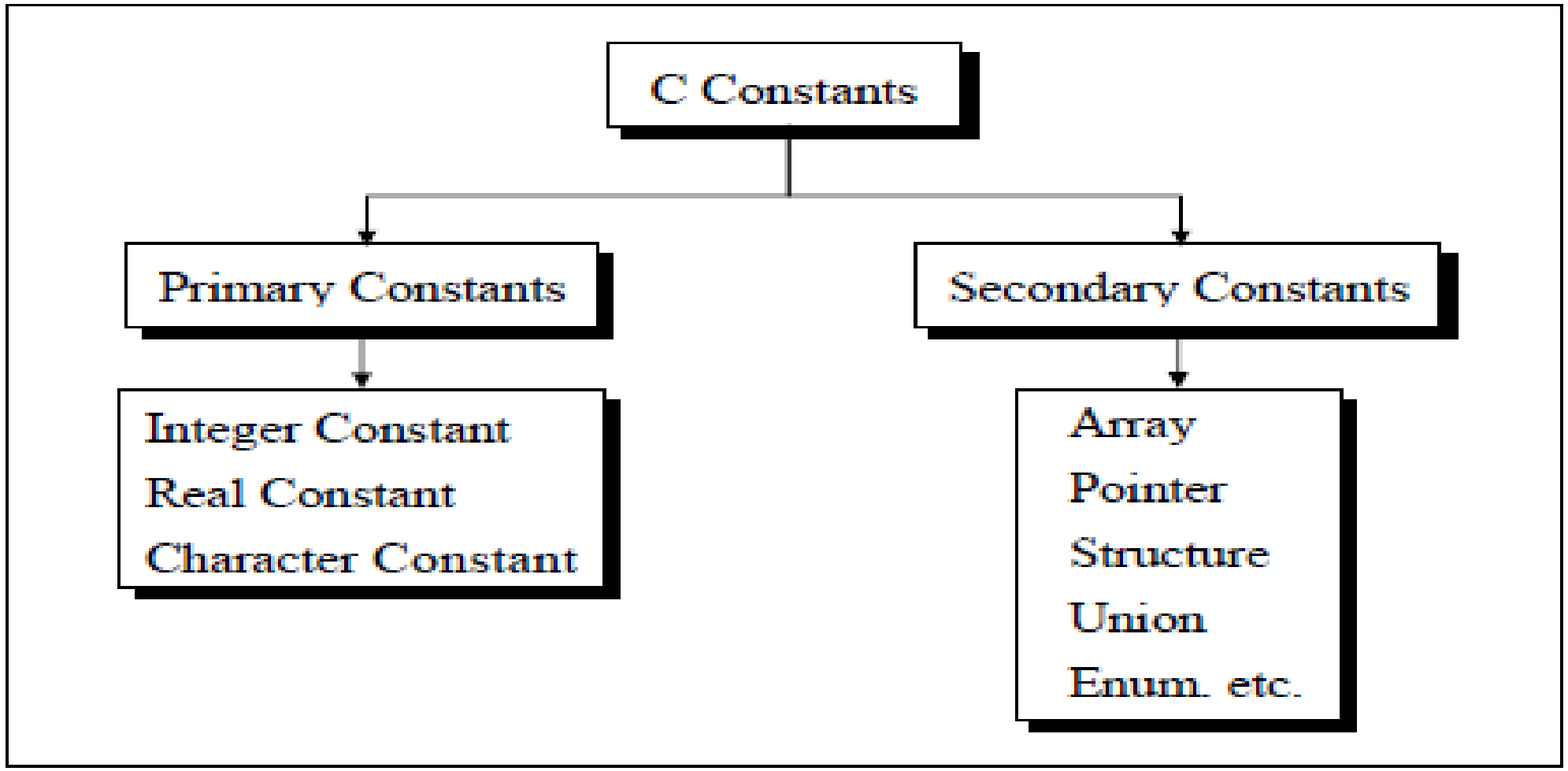
- Steps to learn Natural Language
- Alphabets
- Words
- Sentences
- Paragraphs

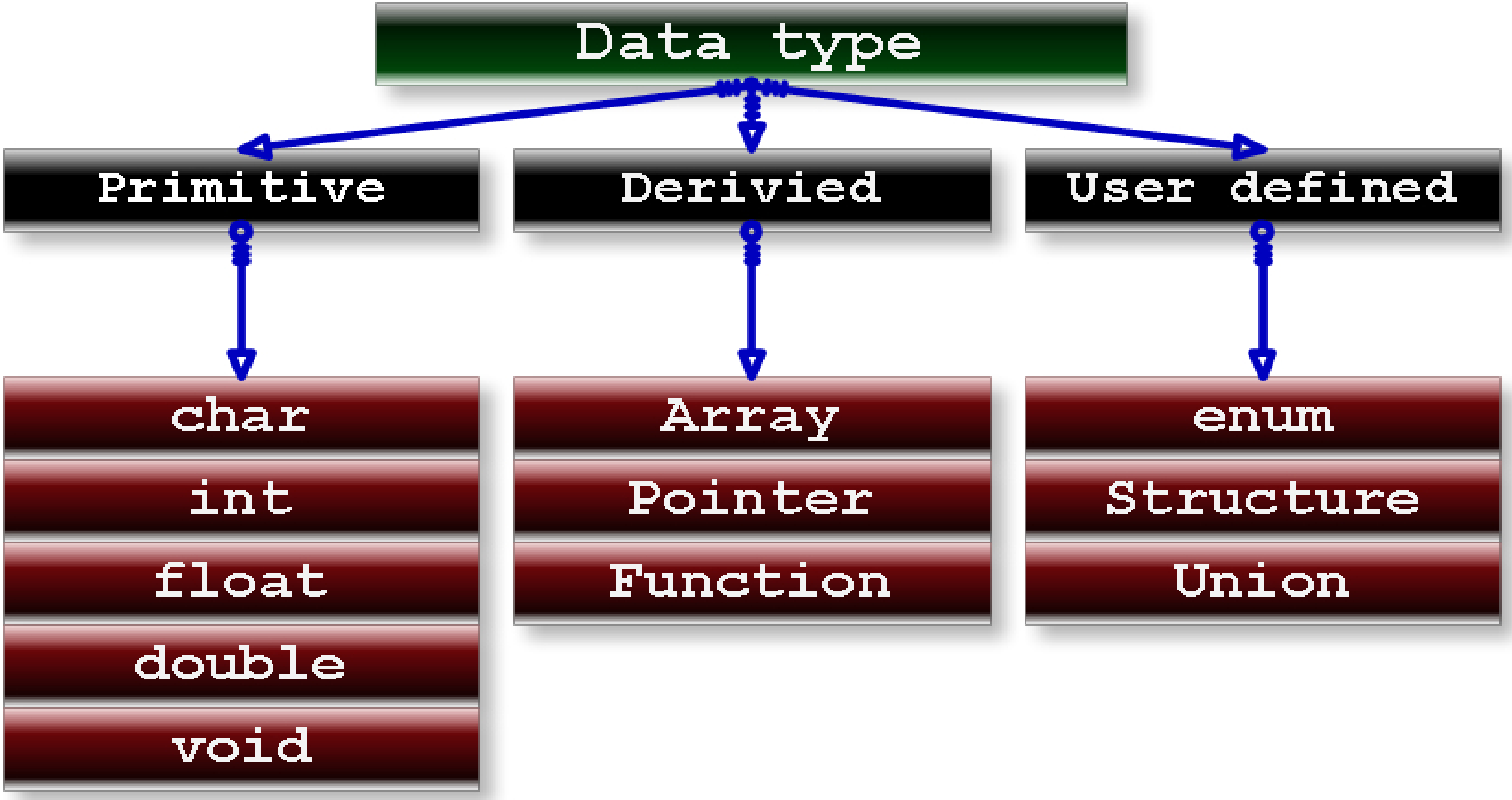
- Steps to learn C language
- Alphabets , digits , special symbols (ascii – 256)
- Constants , variables , keywords
- Instructions
- Program

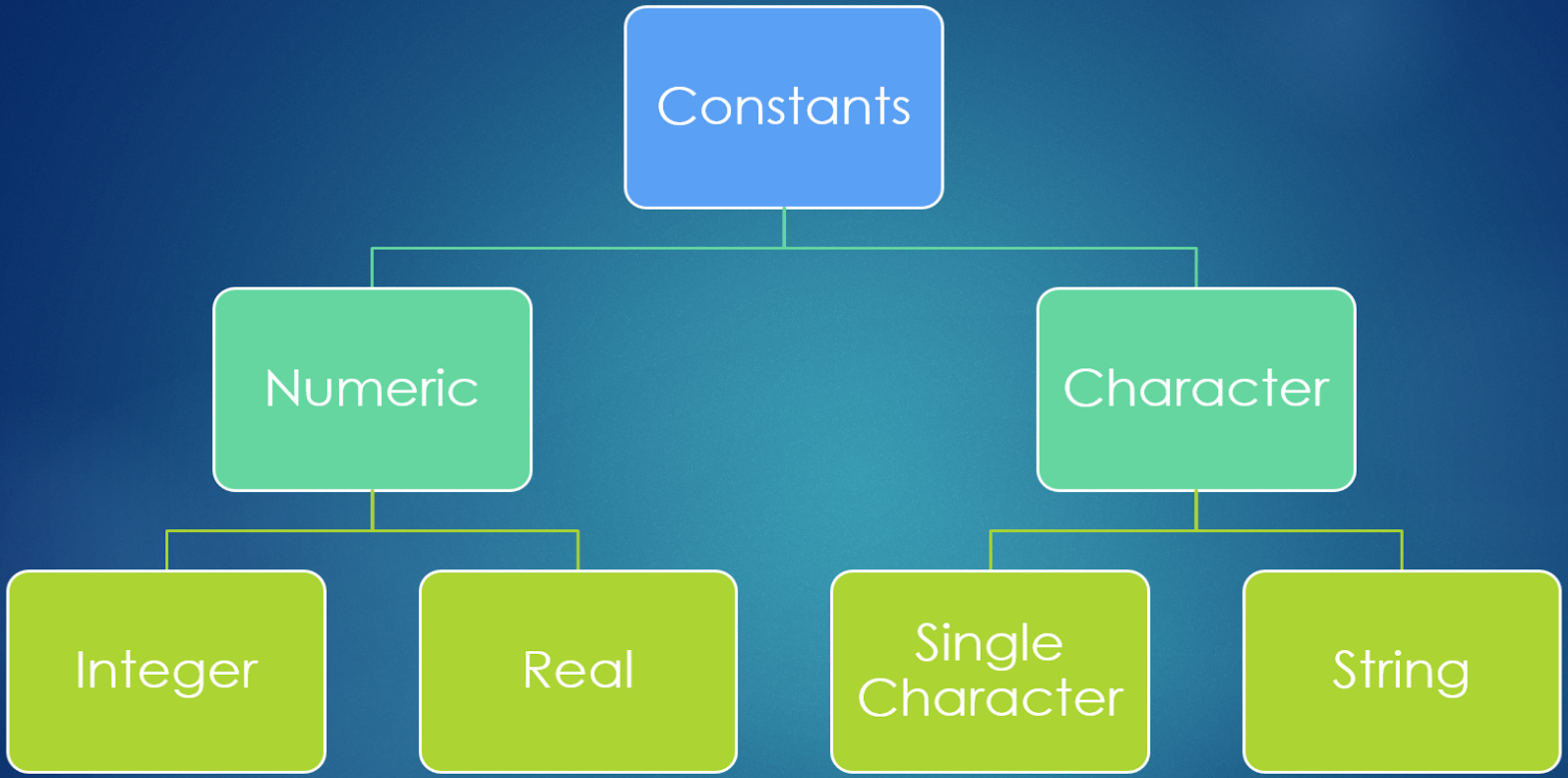
Files in C

- .c Text file
- .Bak file
- .Obj File …. Compiled file Object code file Machine understandable
- .Exe ….. Executable File

- Variables which change their values
- A=9 , B=6 , C=7 : A=b+c;
- Constants in C




Data type
Primitive
Derivied
User defined
char
int
float
double
void
Array
Pointer
Function
enum
Structure
Union



Basic Types of Constants

Data type	Size(bytes)	Range	Format String
char	1	-128 to 127	%c
unsigned char	1	0 to 255	%c
short	2	-32,768 to 32,767	%d
unsigned short	2	0 to 65535	%u
int	2	32,768 to 32,767	%d
unsigned int	2	0 to 65535	%u
long	4	-2147483648 to +2147483647	%ld
Unsinged long	4	0 to 4294967295	%lu
float	4	-3.4e-38 to +3.4e-38	%f
double	8	1.7 e-308 to 1.7 e+308	% lf
long double	10	3.4 e-4932 to 1.1 e+4932	%lf

Keywords in C

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
continue	for	signed	void
do	if	static	while
default	goto	sizeof	Volatile
const	float	short	Unsigned

- Thread
- Process
- Instruction
- Program
- Software

Structure of C Program

Header	#include <stdio.h>
main()	int main() {
Variable declaration	int a = 10;
Body	printf("%d ", a);
Return	return 0; }

Structure of a C program

```
#include <stdio.h>

void main (void)
{
        printf("\nHello World\n");
}
```

Preprocessor directive (header file)

Program statement

```
#include <stdio.h>
#define VALUE 10

int global_var;

void main (void)
{
        /* This is the beginning of the program */

        int local_var;
        local_var = 5;
        global_var = local_var + VALUE;

        printf ("Total sum is: %d\n", global_var); // Print out the result
}
```

Preprocessor directive

Global variable declaration

Comments

Local variable declaration

Variable definition

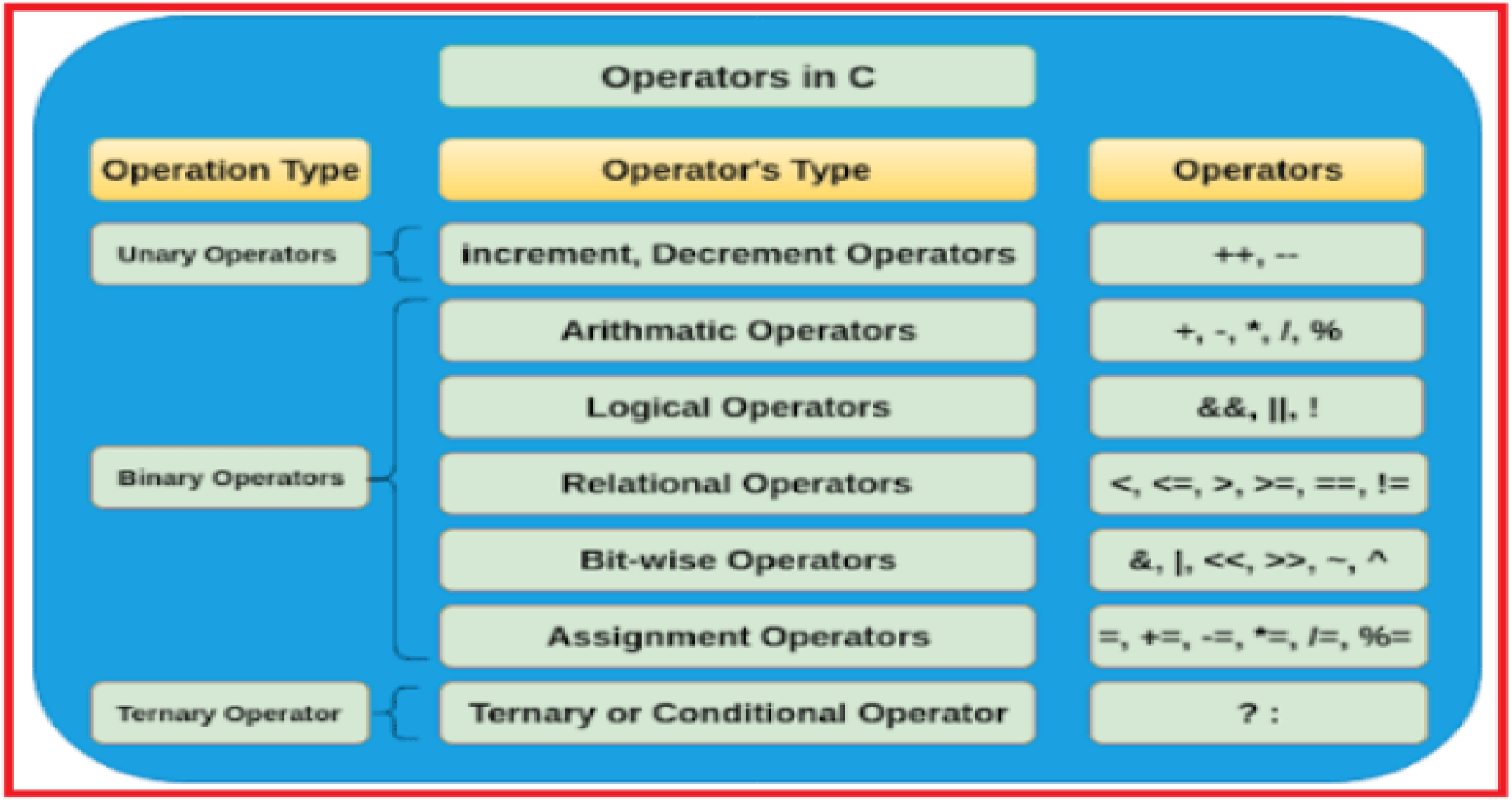
Header Files

- Stdio.h
- Conio.h
- Stdlib.h
- Math.h
- etc

Hello World

- #include<stdio.h>
- Libraries
- General Syntax
- Editors
- Versions

Operators in C



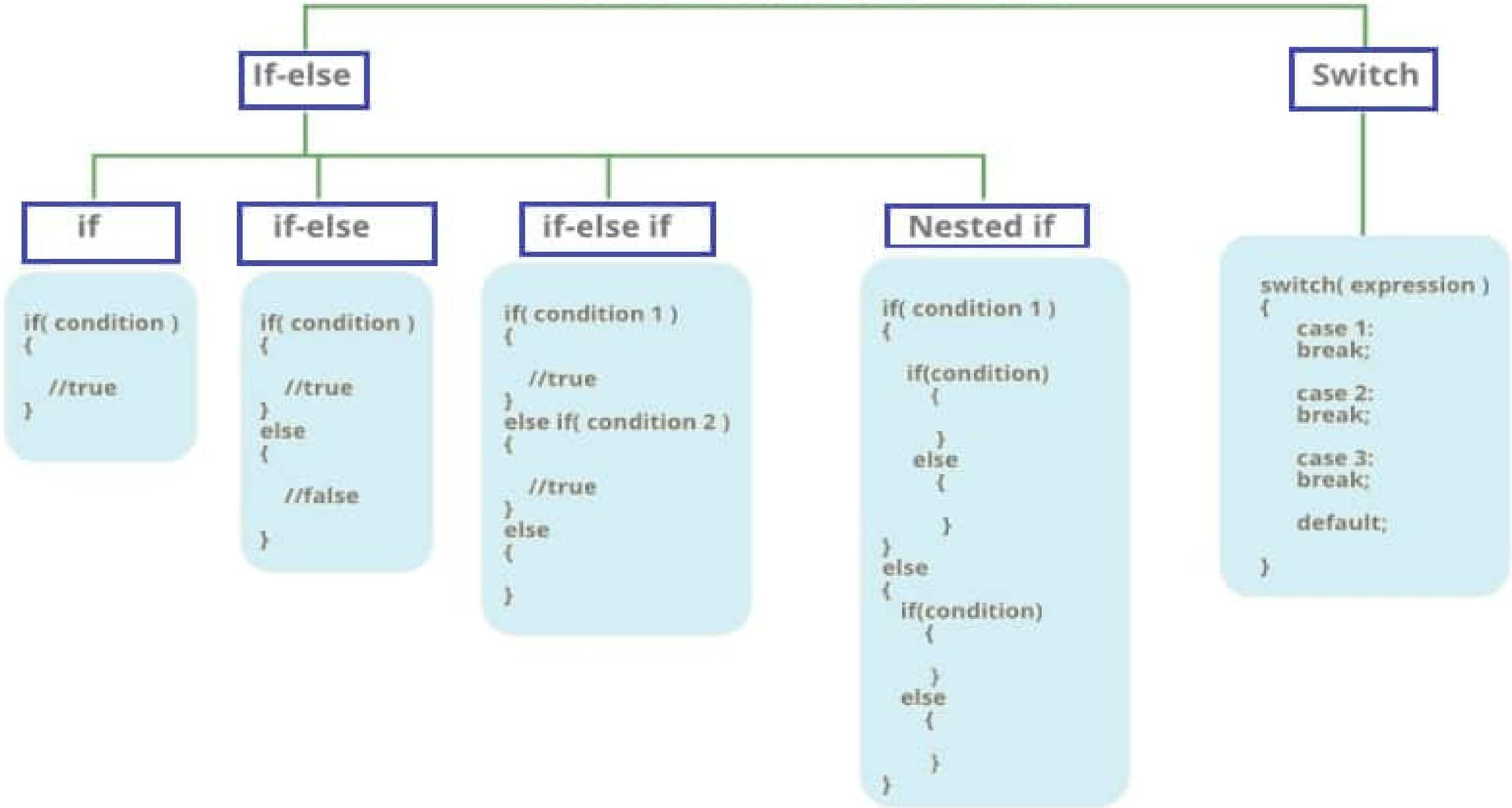
Input Output in C

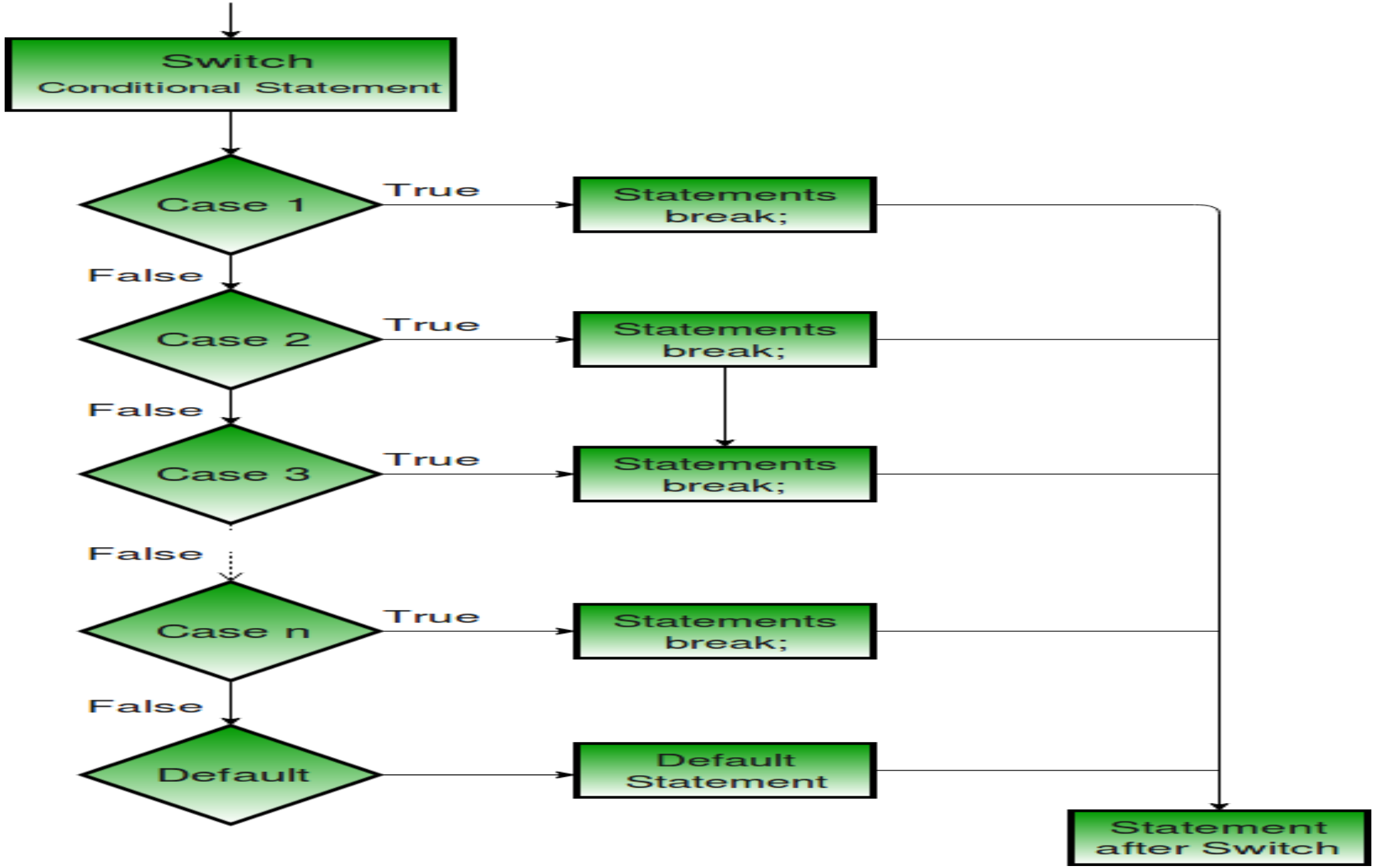
- Printf(“ “);
- Printf(“”,variable);
- Scanf(“ % …. ” , &variable);
- getchar();

Decision making and Branching

- If
- Else
- Nested if
- Nested if else

Conditional Statement




Switch
Conditional Statement
Case 1
True
Statements
break;
False
Case 2
True
Statements
break;
False
Case 3
True
Statements
break;
False
Case n
True
Statements
break;
False
Default
Default
Statement
Statement
after Switch

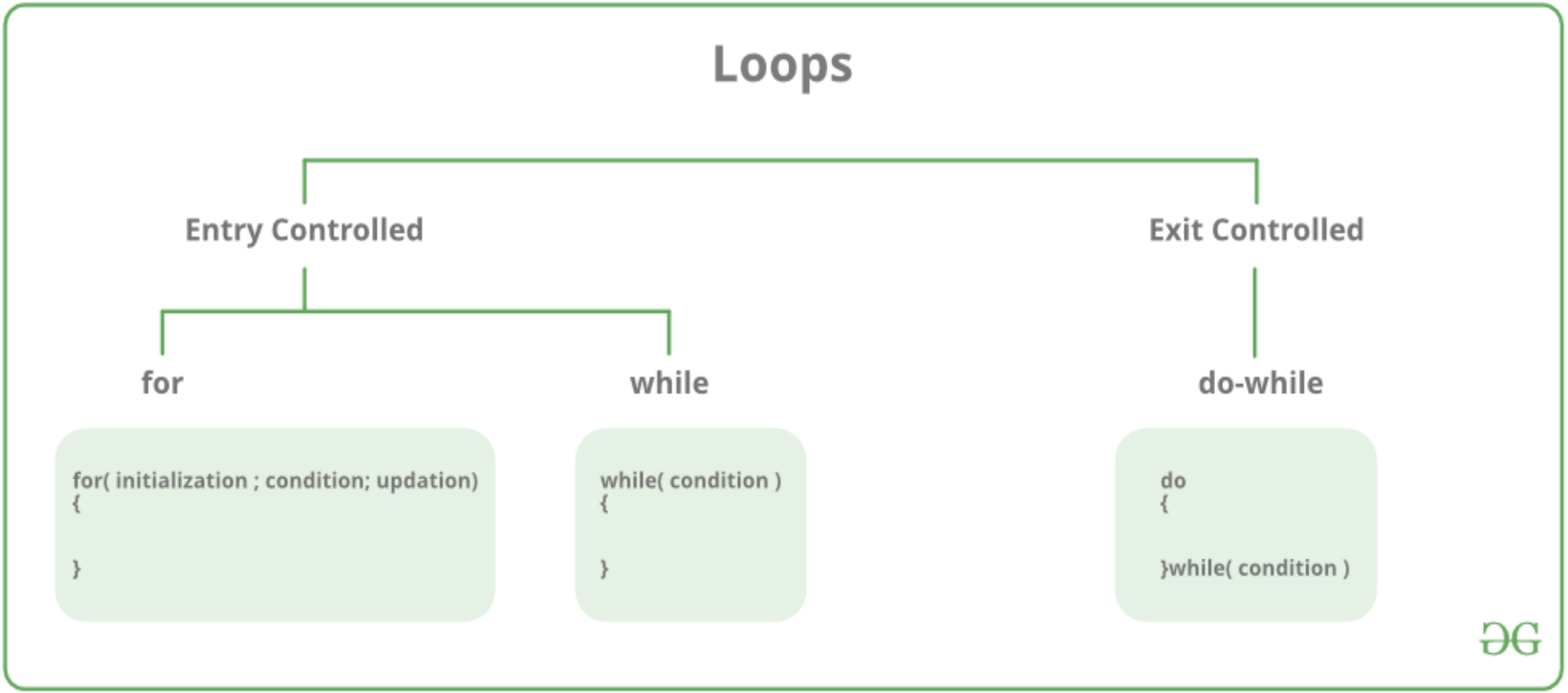
```c
#include <stdio.h>
int main() {
    int num = 8;
    switch (num) {
        case 7:
            printf("Value is 7");
            break;
        case 8:
            printf("Value is 8");
            break;
        case 9:
            printf("Value is 9");
            break;
        default:
            printf("Out of range");
            break;
    }
    return 0;
}
```

1

2

3

Loops in C and their advantages



do-while Loop

While

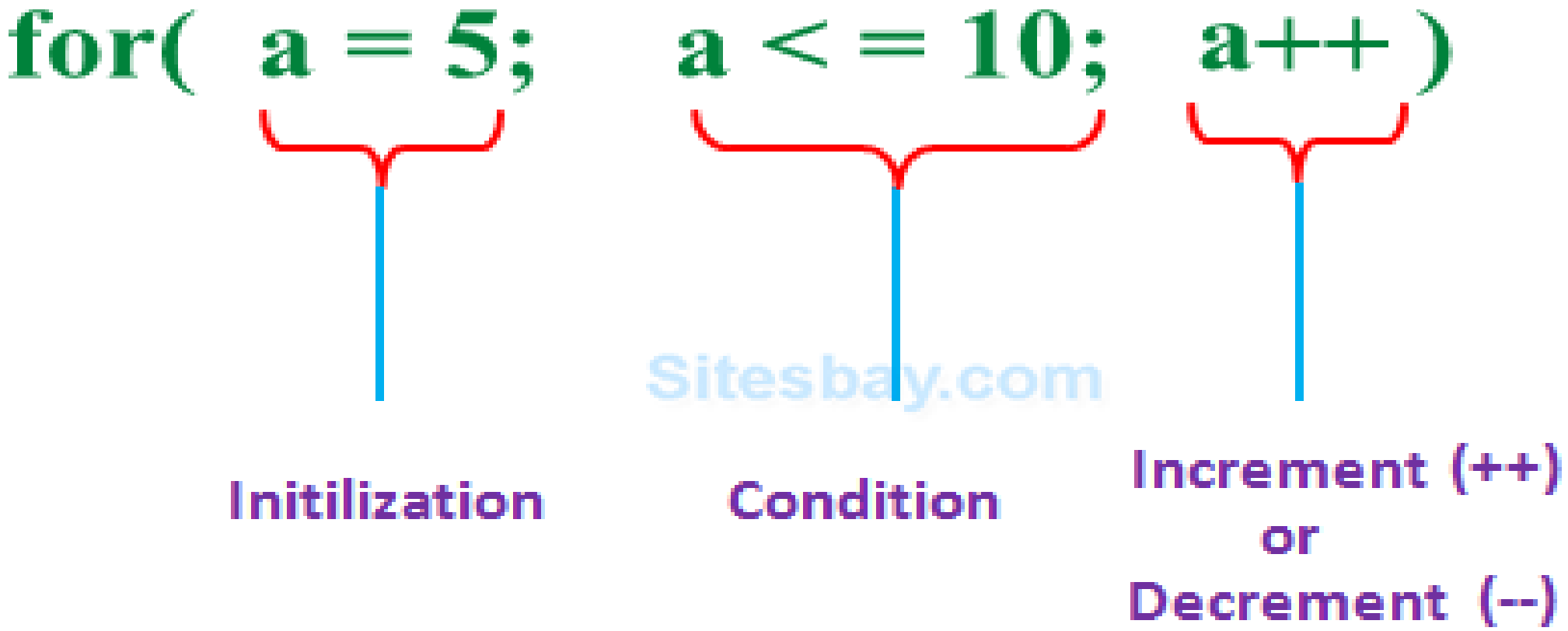
```
int i = 0;
while(i > 0)
{
    printf("%d", i);
    i--;
}
```

do-While

```
int i = 0;
do
{
    printf("%d", i);
    i--;
} while(i > 0);
```

41 C Programming

“For” loop


for(a = 5; a <= 10; a++)
Initilization
Condition
Increment (++)
or
Decrement (--)

- # Break loop

- out of a **loop**.

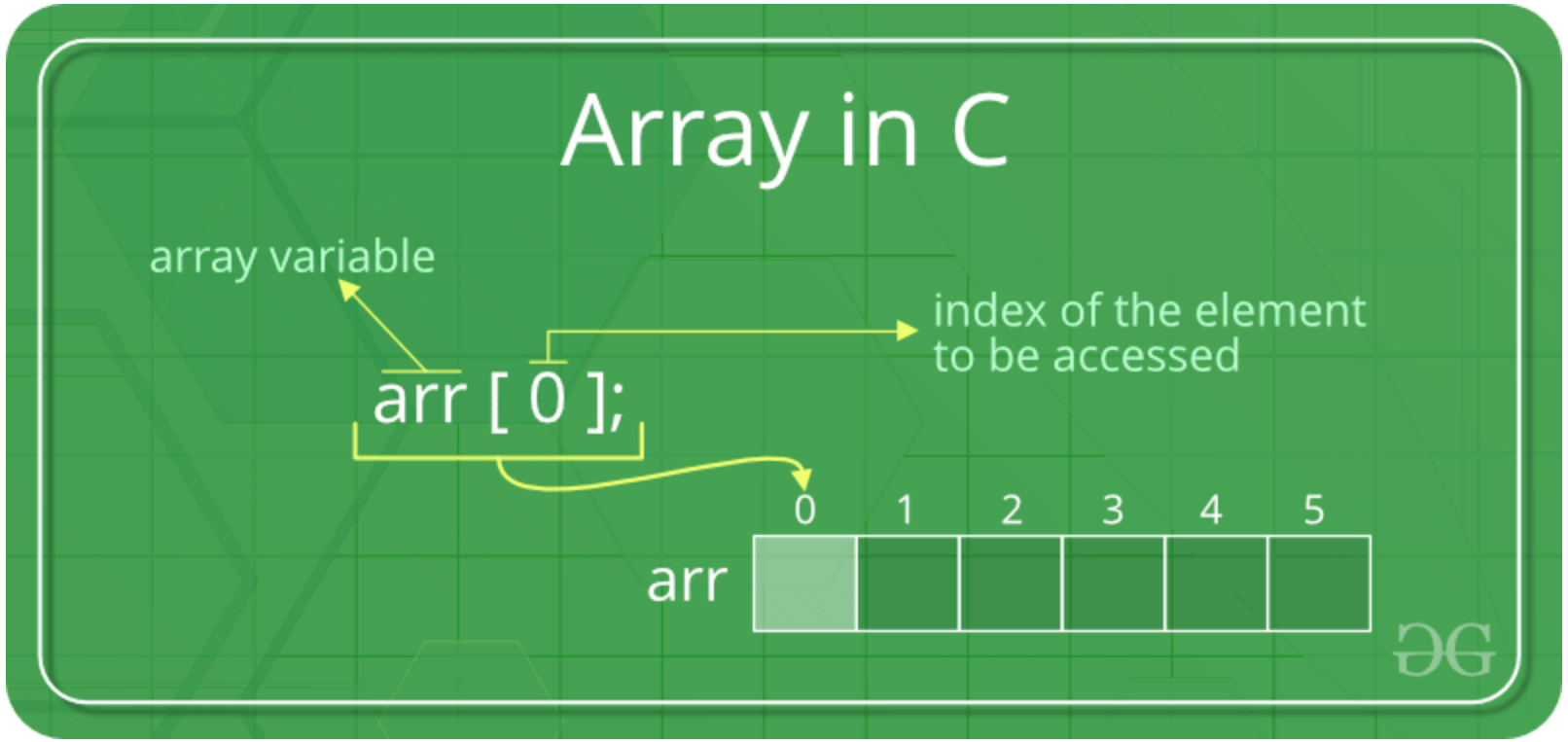
- ```
  for (i = 0; i < 10; i++) {
    if (i == 4) {
      break;
    }
    printf("%d\n", i);
  }
  ```

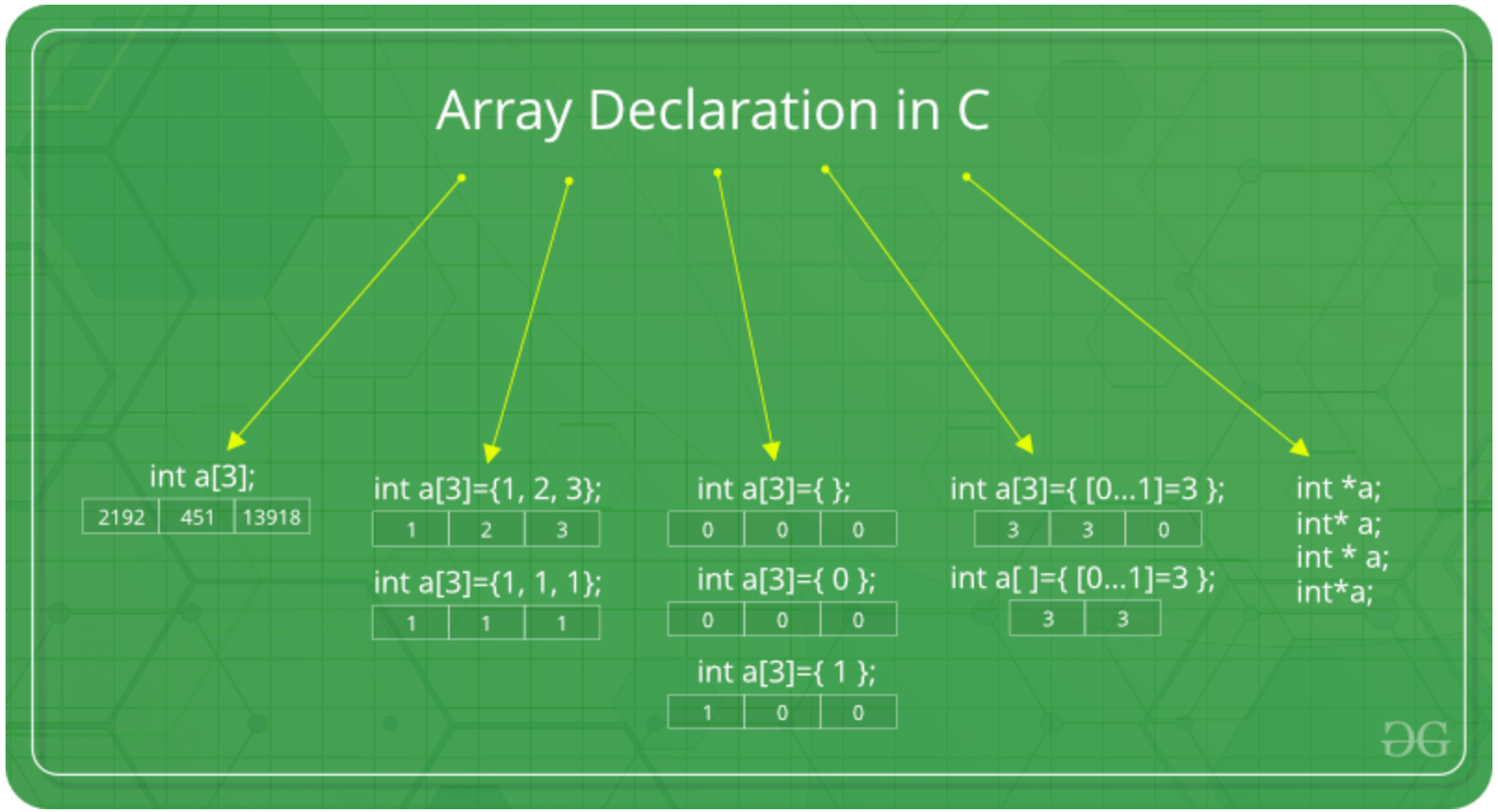
- # Continue loop

breaks one iteration

```
for (i = 0; i < 10; i++) {
  if (i == 4) {
    continue;
  }
  printf("%d\n", i);
}
```
```

Arrays in C

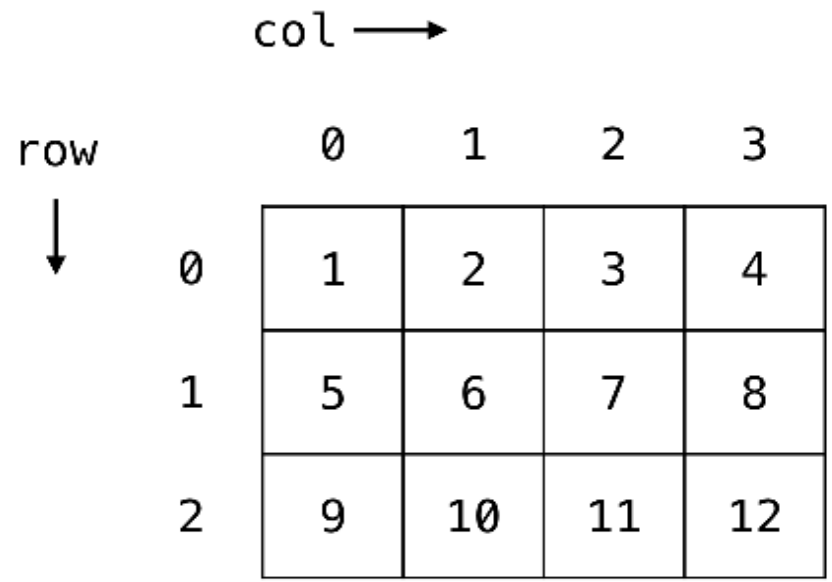

Array in C
array variable
index of the element
to be accessed
arr [0];
0
1
2
3
4
5
arr


Array Declaration in C
int a[3];
2192 451 13918
int a[3]={1, 2, 3};
1 2 3
int a[3]={1, 1, 1};
1 1 1
int a[3]={ };
0 0 0
int a[3]={ 0 };
0 0 0
int a[3]={ 1 };
1 0 0
int a[3]={ [0...1]=3 };
3 3 0
int a[ ]={ [0...1]=3 };
3 3
int *a;
int* a;
int * a;
int*a;

2D arrays

```
int num[3][4] = {
  {1, 2,  3,  4},
  {5, 6,  7,  8},
  {9, 10, 11, 12}
};
```

CLASSROOM



Strings

Initialization of strings

- In C, string can be initialized in a number of different ways.
- For convenience and ease, both initialization and declaration are done in the same step.

```
char c[] = "abcd";
      OR,
char c[50] = "abcd";
      OR,
char c[] = {'a', 'b', 'c', 'd', '\0'};
      OR,
char c[5] = {'a', 'b', 'c', 'd', '\0'};
```

String in c

Important String Functions

1. strlen()
2. strupr()
3. strlwr()
4. strrev()
5. strcpy()
6. strcat()
7. strcmp()

strlen - Finds out the length of a string
strlwr - It converts a string to lowercase
strupr - It converts a string to uppercase
strcat - It appends one string at the end of another
strncat - It appends first n characters of a string at the end of another.
strcpy - Use it for Copying a string into another
strncpy - It copies first n characters of one string into another
strcmp - It compares two strings
strncmp - It compares first n characters of two strings
strcmpi - It compares two strings without regard to case ("i" denotes that this function ignores case)
stricmp - It compares two strings without regard to case (identical to strcmpi)
strnicmp - It compares first n characters of two strings, Its not case sensitive
strdup - Used for Duplicating a string
strchr - Finds out first occurrence of a given character in a string
strrchr - Finds out last occurrence of a given character in a string
strstr - Finds first occurrence of a given string in another string
strset - It sets all characters of string to a given character
strnset - It sets first n characters of a string to a given character
strrev - It Reverses a string

Structure in C

- A structure is a user-defined data type available in C that allows to combining data items of different kinds. Structures are used to represent a record.

```
struct employee
{
    int id;
    char name[50];
    string department;
    string email;
};
```

```
struct student
{
int rollno;
char name[50];
string phone;
};
```

A Union is a user-defined data type. It is just like the structure except that all of its members start at the same location in memory. The union combines various objects of different data types in the same memory location. A user can define a union with many members, but at any given time, only one member can hold a value. The storage space allocated for the union variable is equal to the total space required by the largest data member of the union.

```
union abc
{
   int a;
char b;
}var;
int main()
{
  var.a = 66;
  printf("\n a = %d", var.a);
  printf("\n b = %d", var.b);
}
```

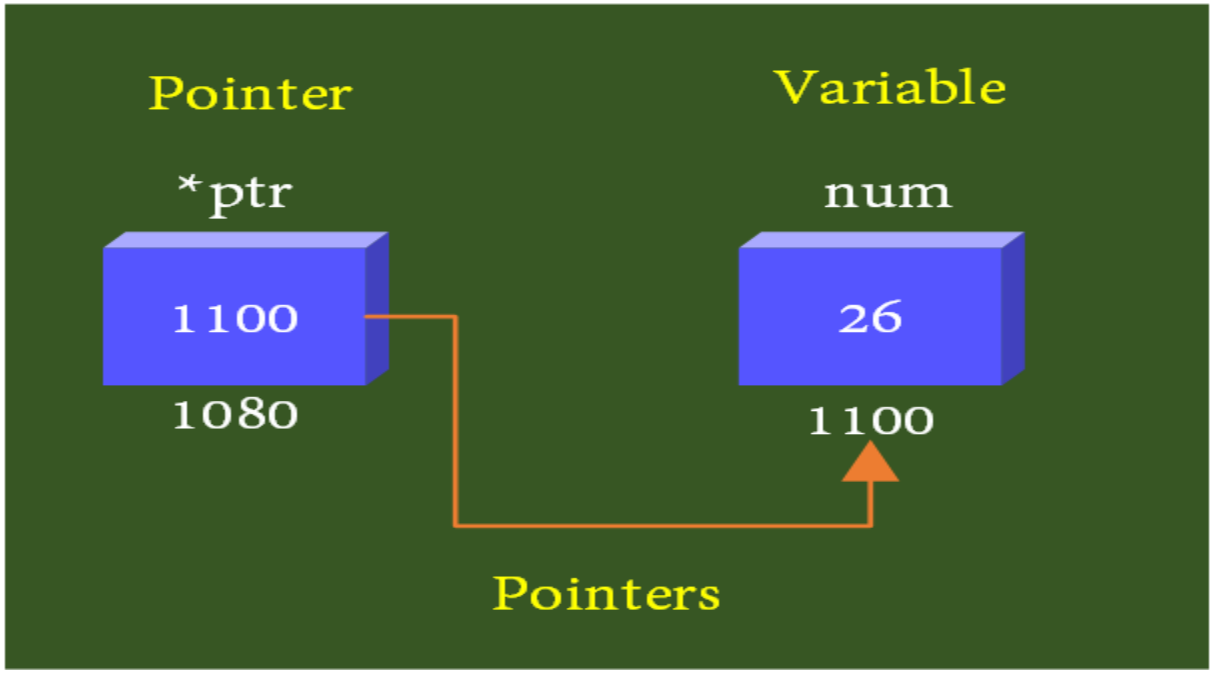
- **Similarities between Structure and Union**

1. Both are user-defined data types used to store data of different types as a single unit.
2. Their members can be objects of any type, including other structures and unions or arrays. A member can also consist of a bit field.
3. Both structures and unions support only assignment = and sizeof operators. The two structures or unions in the assignment must have the same members and member types.
4. A structure or a union can be passed by value to functions and returned by value by functions. The argument must have the same type as the function parameter. A structure or union is passed by value just like a scalar variable as a corresponding parameter.
5. '.' operator is used for accessing members.

	STRUCTURE	UNION
Keyword	The keyword **struct** is used to define a structure	The keyword **union** is used to define a union.
Size	When a variable is associated with a structure, the compiler allocates the memory for each member. The size of structure is **greater than or equal to the sum of sizes of its members.**	when a variable is associated with a union, the compiler allocates the memory by considering the size of the largest memory. So, size of **union is equal to the size of largest member.**
Memory	Each member within a structure is assigned unique storage area of location.	Memory allocated is shared by individual members of union.
Value Altering	Altering the value of a member will not affect other members of the structure.	Altering the value of any of the member will alter other member values.
Accessing members	Individual member can be accessed at a time.	Only one member can be accessed at a time.
Initialization of Members	Several members of a structure can initialize at once.	Only the first member of a union can be initialized.

Pointers in C

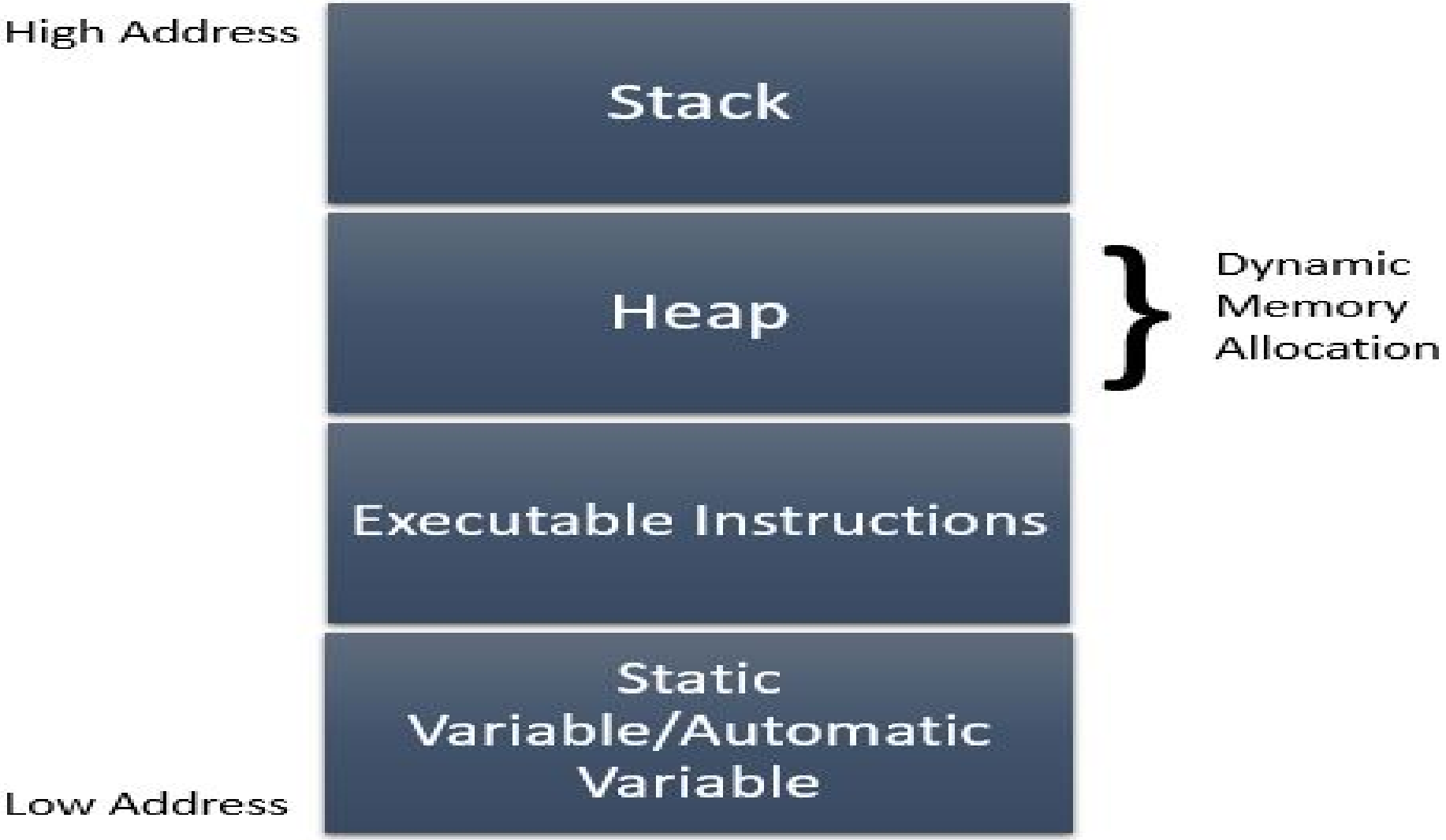
- The pointer in C language is **a variable which stores the address of another variable**. This variable can be of type int, char, array, function, or any other pointer. The size of the pointer depends on the architecture. However, in 32-bit architecture the size of a pointer is 2 byte.



Why are Pointers Used ?

- To return more than one value from a function
- To pass arrays & strings more conveniently from one function to another
- To manipulate arrays more easily by moving pointers to them, Instead of moving the arrays themselves
- To allocate memory and access it (Dynamic Memory Allocation)
- To create complex data structures such as Linked List, Where one data structure must contain references to other data structures

Memory Management In C



Function	Task
malloc( )	Allocate request size of bytes & return a pointer to the first byte of the allocated space. And contains garbage values.
calloc( )	Allocate space for an array of elements, initialize them to zero and returns a pointer to the first byte of allocated space.
realloc( )	Modify the size of previously allocated space.
free( )	Free the previously allocated space.

File handling in C

File operation	Declaration & Description
fopen() – To open a file	Declaration: FILE *fopen (const char *filename, const char *mode) fopen() function is used to open a file to perform operations such as reading, writing etc. In a C program, we declare a file pointer and use fopen() as below. fopen() function creates a new file if the mentioned file name does not exist. FILE *fp; fp=fopen (“filename”, “‘mode”); Where, fp – file pointer to the data type “FILE”. filename – the actual file name with full path of the file. mode – refers to the operation that will be performed on the file. Example: r, w, a, r+, w+ and a+. Please refer below the description for these mode of operations.
fclose() – To close a file	Declaration: int fclose(FILE *fp); fclose() function closes the file that is being pointed by file pointer fp. In a C program, we close a file as below. fclose (fp);
fgets() – To read a file	Declaration: char *fgets(char *string, int n, FILE *fp) fgets function is used to read a file line by line. In a C program, we use fgets function as below. fgets (buffer, size, fp); where, buffer – buffer to put the data in. size – size of the buffer fp – file pointer
fprintf() – To write into a file	Declaration: int fprintf(FILE *fp, const char *format, ...);fprintf() function writes string into a file pointed by fp. In a C program, we write string into a file as below. fprintf (fp, “some data”); or fprintf (fp, “text %d”, variable_name);

Function	description
fopen()	create a new file or open a existing file
fclose()	closes a file
getc()	reads a character from a file
putc()	writes a character to a file
fscanf()	reads a set of data from a file
fprintf()	writes a set of data to a file
getw()	reads a integer from a file
putw()	writes a integer to a file
fseek()	set the position to desire point
ftell()	gives current position in the file
rewind()	set the position to the begining point